

RALLYSTATS · DOCUMENTO INTERNO

Roadmap de funcionalidades premium

Qué se necesitaría construir para cada funcionalidad candidata a justificar la suscripción mensual, ancladas al código real de la app (modelos, servicios, pantallas), con esfuerzo estimado y riesgos de producto.

Preparado como insumo de decisión, no como plan de implementación cerrado.
30 de agosto de 2026

Contenido

0. Cómo se gatea premium hoy (patrón a reutilizar)
1. Panel de temporada (estadísticas acumuladas multi-partido)
2. Scouting de rival (histórico contra un mismo equipo)
3. Estadísticas por rotación (K1-K6)
4. Exportar estadísticas a Excel/CSV
5. Backup / sincronización en la nube de equipos y partidos
6. Fixture: calendario de partidos con autocompletado
7. Chatbot conversacional (IA) sobre los datos del partido
8. Otras funcionalidades adicionales
9. Orden sugerido de implementación

Documento técnico de referencia: para cada funcionalidad propuesta para justificar la suscripción mensual se detalla el objetivo, cómo se resolvería contra la arquitectura actual del código (modelos, servicios, pantallas), qué archivos nuevos o modificados implica, cómo se gatearía detrás de premium, una estimación de complejidad y los riesgos o decisiones de producto pendientes. Generado a partir de una revisión del código real (lib/models, lib/services, lib/state, lib/screens) y, para la sección de chatbot, del análisis previo Analisis-Chatbot-RallyStats.pdf - antes de implementar cualquier punto conviene revalidar que la parte del código citada no haya cambiado desde entonces.

0. Cómo se gatea premium hoy (patrón a reutilizar)

No hay un único gate central de funcionalidad (a diferencia del login, que sí tiene un gate único en `_AuthGate`). Cada restricción se resuelve en el punto de uso, contra `SubscriptionController.isPremium`, con dos formas que ya conviven en el código:

- **Bloqueo duro con excepción:** `AppDataController.saveMatch` tira `MatchArchiveLimitException` si `!isPremium && matches.length >= 3`, y el llamador (`MatchArchiveScreen._importMatch`) atrapa esa excepción para mostrar el paywall (`showRallyStatsPaywall`, en `paywall_launcher.dart`).
- **Límite de UI en línea:** `NewMatchScreen._maxSetsAllowed` lee `context.read<SubscriptionController>().isPremium` directo en el widget y limita el máximo de sets a 3 en vez de 5.

Cualquier funcionalidad nueva de este documento debería seguir uno de estos dos patrones según corresponda: excepción + paywall si es una acción puntual (guardar, exportar, crear), o chequeo en línea si es un límite de cantidad/alcance dentro de una pantalla ya abierta. No hace falta inventar un mecanismo nuevo de gating.

1. Panel de temporada (estadísticas acumuladas multi-partido)

Objetivo

Ver la evolución de un jugador o del equipo a lo largo de todos los partidos guardados (o filtrados por torneo/rango de fechas), no solo dentro de un partido como hoy hace `MatchSummaryScreen`.

Cómo se resolvería

`StatsEngine.compute()` ya es puramente derivado de `List<RallyEvent>` y ya tiene las funciones de acumulación que hacen falta (`_accumulate`, `_accumulateRecepcion`) para sumar `TouchStats/ReceptionStats` de varias fuentes. No hace falta tocar el modelo de datos ni la persistencia.

- Nuevo método `StatsEngine.computeSeason(List<VolleyMatch> matches)` (o un archivo nuevo `season_stats_engine.dart`): llama `StatsEngine.compute(match)` para cada partido de la lista filtrada y suma los `PlayerStatLine` resultantes por `playerId`, reutilizando `_accumulate/_accumulateRecepcion` (hoy privados a `stats_engine.dart`; conviene dejarlos como funciones de nivel de archivo reusables en vez de duplicarlas).
- Filtro de partidos por `ownTeamSourceId` (para elegir "mi equipo"), con filtros opcionales de rango de fechas y `tournament`. La lista completa sale de `AppDataController.matches`.
- Nueva pantalla `lib/screens/matches/season_stats_screen.dart`: selector de equipo propio + filtros, y debajo la misma tabla de estadísticas que ya usa `MatchSummaryScreen` (conviene extraer esa tabla a un widget compartido si hoy está inline, en vez de duplicar el layout).

- Punto de entrada: un botón nuevo en MatchArchiveScreen que abre la pantalla; ahí se hace el chequeo `isPremium`; si no, `showRallyStatsPaywall(context)`.

Consideraciones

- `ownRoster` es una copia de los jugadores en cada `VolleyMatch` (no una referencia viva a `Team`): para agrupar hay que usar `Player.id` como clave estable, y decidir una política simple para mostrar nombre/dorsal si cambió entre partidos (por ejemplo, el del partido más reciente).
- Filtrar siempre por `ownTeamSourceId`, no solo por nombre, para no mezclar jugadores homónimos de otro club.

Complejidad estimada

Media. Sin cambios de persistencia; el trabajo es agregación + una pantalla nueva reusando UI existente.

2. Scouting de rival (histórico contra un mismo equipo)

Objetivo

Cuando se va a jugar contra un rival ya enfrentado antes, ver cómo sumó y perdió puntos ese rival en los partidos anteriores, para preparar el partido.

Fase 1 - con los datos que ya existen

`StatsEngine.compute()` ya devuelve `RivalErrorStats` y `RivalPointStats`, desglosados en saque/ataque/contra. Se puede armar `StatsEngine.computeRivalScouting(List<VolleyMatch> matches)`, análogo al panel de temporada, filtrando por `rivalTeamName` (o mejor, por `rivalTeamSourceId` cuando esté seteado - más confiable que comparar texto libre; ver la sección de Fixture, que ayuda justamente a que el rival quede identificado por id). Nueva pantalla `lib/screens/matches/rival_scouting_screen.dart`, accesible desde `MatchArchiveScreen` o desde el armado de un partido nuevo.

Fase 2 - mapa de zonas de ataque rival (no recomendada para una primera versión)

Hoy `computeZones()` solo calcula zona de destino para el equipo propio - no se captura la zona de ataque del rival en absoluto. Para un scouting con mapa de zonas rival haría falta (a) una forma de cargar en vivo con qué zona atacó el rival, y (b) extender `RallyEvent` con un campo `rivalTargetZone`. Es un cambio de captura de datos en vivo, no solo de reporte - mucho más trabajo que la fase 1. Dejarlo como iteración futura, después de validar que la fase 1 aporta valor real.

Complejidad estimada

Fase 1: media (idéntica a la del panel de temporada, mismo motor de agregación). Fase 2: alta - no dimensionar junto con el resto del roadmap.

3. Estadísticas por rotación (K1-K6)

Objetivo

Eficiencia de saque/recepción/ataque desglosada por la rotación en la que estaba el equipo en ese momento, para decidir el sistema de juego.

Por qué no es tan directo como zona de destino

`MatchSet.currentOrderOwn` es el estado de rotación *actual*, mutable con cada rotación y cada cambio de jugador - no queda un historial de en qué rotación estaba el equipo en cada rally. `RallyEvent` no tiene ese dato. La única forma de recuperarlo hoy sería re-simular todo el log de eventos con la misma lógica de `MatchController`, lo cual duplica lógica de reglas del juego en el motor de estadísticas.

Cómo se resolvería (mismo patrón que `targetZone`)

Agregar un campo opcional nuevo a `RallyEvent`, por ejemplo `rotationSlot` (1-6: la posición de rotación del equipo propio en el momento del evento), análogo a cómo ya se guarda `targetZone` o `servingTeamBefore`. Se completa en `MatchController`, en el mismo punto donde hoy arma `ownScoreAfter/rivalScoreAfter` al crear cada `RallyEvent`, leyendo la posición 1 (saque) de `MatchSet.currentOrderOwn` en ese instante. Partidos guardados antes de este cambio simplemente no tendrán desglose por rotación (`rotationSlot` queda null), siguiendo el mismo patrón de migración que ya usan `targetZone/rivalActionType`.

- Modificar: `lib/models/rally_event.dart` (campo + `toJson/fromJson`) y `lib/state/match_controller.dart` (poblarlo al crear cada evento).
- Nuevo: `StatsEngine.computeByRotation(VolleyMatch match, {int? setNumber})`, análogo a `computeZones()` pero agrupando por `rotationSlot`.
- Nueva UI: una pestaña o sección más en `MatchSummaryScreen`, con una tabla de 6 columnas (K1..K6).

Consideraciones

A diferencia de las dos secciones anteriores, esta sí toca `match_controller.dart` (el "corazón" de la app) y el formato persistido de `RallyEvent` - conviene hacerla con más cuidado y con partidos de prueba completos, no en la misma pasada que las funcionalidades puramente de reporte.

Complejidad estimada

Media-alta (toca captura en vivo, no solo reporte).

4. Exportar estadísticas a Excel/CSV

Objetivo

Que el cuerpo técnico pueda llevarse las estadísticas a una planilla propia, más allá del PDF de solo lectura que ya genera `pdf_report_service.dart`.

Cómo se resolvería

Nuevo `lib/services/csv_export_service.dart`, calcado del patrón ya usado en `match_export_service.dart` (mismo uso de `share_plus` en Android/iOS y `file_picker` en desktop para elegir dónde guardar). Contenido: recorrer `MatchStats.orderedRows` (o el resultado de `computeSeason/computeRivalScouting` si se exportan esos paneles también) y armar líneas CSV a mano - no hace falta un paquete de CSV, es un formato simple de valores separados por coma. Se puede ofrecer desde el mismo lugar de donde hoy se comparte/exporta el PDF.

Gating

Mismo criterio que el resto de "Estadísticas" en la versión actual - el botón de exportar CSV vive dentro de una pantalla que ya está gateada, así que alcanza con que la pantalla contenedora ya chequee `isPremium`.

Complejidad estimada

Baja. Es el punto más rápido de todo este documento.

5. Backup / sincronización en la nube de equipos y partidos

Objetivo

Que un entrenador no pierda su archivo de partidos/equipos si cambia de dispositivo, sin depender de exportar/importar .json a mano partido por partido (que es lo único que existe hoy vía `MatchExportService`).

Por qué es el punto de mayor impacto arquitectónico de este documento

Hoy Firebase sólo se usa para autenticación y para el conteo de dispositivos (`account_devices` en Firestore) - todos los datos de juego (equipos, partidos, jugadas de pizarra) son 100% locales en Hive, por diseño explícito del proyecto. Meter un backup en la nube no es solo agregar una llamada a Firestore: es una decisión de producto que cambia esa premisa, con implicancias de privacidad (los datos de un partido pasarían a vivir en un servidor) y de costo (Firestore/Storage no son gratis a partir de cierto volumen).

Cómo se resolvería (versión acotada, recomendada para empezar)

Backup manual "on demand", no sincronización automática en tiempo real - evita el problema de resolver conflictos si dos dispositivos editan el mismo partido a la vez (con el modelo actual, un partido es un log de eventos completo, mergearlo automáticamente sería genuinamente complicado):

- Nuevo `lib/services/cloud_backup_service.dart`: sube `Team.toJson()`/`VolleyMatch.toJson()`/`Play.toJson()` a Firebase Storage o Firestore, bajo el uid del usuario logueado (mismo `AuthService.currentUser`).
- Botón "Hacer backup ahora" / "Restaurar desde backup" en una pantalla de configuración, no un sync automático en cada cambio.
- Restaurar reemplaza (o mergea por id, a elección) los datos locales de `StorageService`.

Gating

Premium natural - beneficio de continuidad, similar en espíritu al de multi-dispositivo que ya existe.

Complejidad estimada

Alta, y no solo técnica: antes de tocar código conviene resolver explícitamente la política de conflictos y confirmar el costo esperado en Firebase con el volumen real de usuarios. Es uno de los dos puntos de este documento (junto con el chatbot, sección 7) que no conviene arrancar sin antes conversar el alcance.

6. Fixture: calendario de partidos con autocompletado

Objetivo

Armado el fixture de la temporada/torneo con fecha y rival de cada partido, y que al crear un partido nuevo se pueda elegir una fecha del fixture y que autocomplete los datos (rival, torneo, instancia, cancha, categoría) en vez de tipearlos de nuevo.

Modelo de datos nuevo

`lib/models/fixture_entry.dart`, con el mismo patrón manual de `toJson/fromJson` que ya usan `Team/VolleyMatch/Play` (sin `build_runner`, por la misma razón documentada en `CLAUDE.md`):

```
class FixtureEntry {
  final String id;
  DateTime date;
  String tournament;
  String round;
  String court;
  String category;
  String rivalTeamName;
  String? rivalTeamSourceId; // igual que VolleyMatch.rivalTeamSourceId
  String? notes;
  String? linkedMatchId; // id del VolleyMatch creado a partir de esta fila
  // toJson / fromJson análogos a Team/VolleyMatch
}
```

Persistencia

Nueva Hive box en `lib/services/storage_service.dart`, calcada de las de `Team/Play` (`_fixtureBox = 'fixture_box'`, `loadFixtureEntries()`, `saveFixtureEntry()`, `deleteFixtureEntry()`).

`AppDataController` suma un `List<FixtureEntry>` `fixtureEntries` cargado en `loadAll()`, con sus métodos de alta/baja/edición, igual que ya hace con `teams/matches/plays`.

Pantallas nuevas (`lib/screens/fixture/`)

- `fixture_screen.dart`: lista de partidos programados, ordenados por fecha (patrón calcado de `MatchArchiveScreen/PlaysArchiveScreen`). Cada fila muestra fecha, rival, torneo/instancia; si `linkedMatchId != null` se marca como "jugado" y tocar la fila navega directo al `MatchSummaryScreen` de ese partido en vez de abrir el formulario de edición.
- `fixture_entry_form_screen.dart`: alta/edición de una fila, con los mismos campos que ya pide `NewMatchScreen` hoy (fecha vía `showDatePicker`, torneo, instancia, cancha, categoría, y rival con el mismo combo de "nombre libre + Team precargado opcional" que ya tiene `new_match_screen.dart`).
- Acceso: un tile nuevo en la pantalla principal (junto a "Equipos" / "Partidos" / "Pizarra"), y un botón "Elegir del fixture" dentro de `NewMatchScreen`.

Autocompletado al armar un partido

Siguiendo el mismo patrón que ya usa `PlaysArchiveScreen` con su parámetro `pickMode` (que cuando está en modo selección hace `Navigator.pop(context, play)` en vez de abrir la jugada), `FixtureScreen` recibe un `pickMode` análogo. En `_NewMatchScreenState` se agrega:

```
Future<void> _pickFromFixture() async {
  final entry = await Navigator.push<FixtureEntry>(
    context,
    MaterialPageRoute(builder: (_) => const FixtureScreen(pickMode: true)),
  );
  if (entry == null) return;
  setState(() {
    _date = entry.date;
    _tournamentCtrl.text = entry.tournament;
    _roundCtrl.text = entry.round;
    _courtCtrl.text = entry.court;
    _categoryCtrl.text = entry.category;
    _rivalNameCtrl.text = entry.rivalTeamName;
    if (entry.rivalTeamSourceId != null) {
      _rivalTeam = teams.firstWhereOrNull(
        (t) => t.id == entry.rivalTeamSourceId;
      )
    }
    _pendingFixtureEntryId = entry.id;
  });
}
```

Es exactamente la misma mecánica que ya existe hoy cuando se elige un Team rival precargado del dropdown y autocompleta `_rivalNameCtrl.text` - no hay concepto nuevo, sólo una fuente de datos adicional para rellenar los mismos campos.

Cerrar el círculo (marcar el fixture como jugado)

Cuando finalmente se guarda el `VolleyMatch` (en el paso donde hoy `RosterScreen/LineupScreen` terminan de armar el partido y lo persisten), si vino de un fixture entry hay que: (a) guardar un `fixtureEntryId` opcional nuevo en `VolleyMatch` (mismo patrón de migración manual que ya documenta CLAUDE.md: null en partidos viejos) y (b) actualizar ese `FixtureEntry.linkedMatchId` con el id del partido recién creado. Esto es lo que le permite a `FixtureScreen` mostrar qué partidos del calendario ya se jugaron.

Decisión de producto pendiente (no técnica)

¿El fixture es gratis o premium? Es una funcionalidad de comodidad/organización, no de estadística - técnicamente se podría dejar liberada para todos como "gancho" de uso frecuente de la app y sólo gatear el archivo de partidos jugados en sí (que ya está gateado). Si se quiere que sume como argumento de suscripción, la forma más simple y consistente sería un tope de filas en la versión free (mismo patrón que `MatchArchiveLimitException`, p. ej. "hasta 5 partidos programados por vez"). Vale la pena decidir esto antes de escribir el gating.

Complejidad estimada

Media. No toca `MatchController` ni las reglas de juego - es CRUD + un flujo de selección adicional en un wizard que ya existe, reutilizando patrones (`pickMode`, Hive box nueva, `toJson/fromJson`) que ya están probados en el resto de la app.

Extensiones a futuro (no para la primera versión)

- Importar el fixture completo desde un archivo que publique la liga, en vez de cargar partido por partido a mano.
- Recordatorio/notificación del próximo partido del fixture - requeriría sumar un plugin de notificaciones locales, hoy no presente en el proyecto.

7. Chatbot conversacional (IA) sobre los datos del partido

Es técnicamente viable, pero no es una feature chica: es la primera vez que datos de partido (hoy 100% locales) tendrían un motivo para salir del dispositivo. La recomendación corta es que no hace falta "entrenar" ningún modelo propio - conviene usar un LLM ya existente (vía API) y darle como contexto los datos que `stats_engine.dart` ya calcula, con un alcance inicial acotado (explicar reglas y estadísticas del propio partido) antes que un chat abierto sin límites.

RallyStats tiene un modelo de datos rico y ya calculado - eventos de rally, estadísticas por jugador, rotaciones, cambios de líbero - que hoy solo se muestra en pantallas fijas (tablas, PDF de reporte). Un chatbot no agrega datos nuevos: agrega una forma distinta de *consultar* los datos que ya existen, en lenguaje natural. Eso es lo que lo hace viable sin tener que construir un modelo de IA propio: el LLM no necesita "saber vóley" de memoria, necesita que le pasemos el partido ya resuelto y responda preguntas sobre eso.

El costo real del proyecto no está en la parte de IA (integrar una API de chat es relativamente simple), sino en las decisiones de producto que trae: si conviene romper el offline-first para esto, quién paga el costo variable por consulta, y cómo evitar que el chatbot invente una estadística o explique mal una regla en medio de un partido real.

Qué se le podría preguntar

Tipo de consulta	Ejemplos	Riesgo
Estadísticas de un partido o jugador	"¿Cuántos puntos de saque directo hizo Martina en el set 3?" / "¿Quién tuvo mejor eficacia de recepción?"	Bajo - el chatbot solo redacta un MatchStats ya calculado, no inventa números.
Comparaciones y tendencias	"Comparará el rendimiento de Juan entre el partido de hoy y el del 12/08"	Bajo-medio - más costoso en tokens (pasa más de un VolleyMatch como contexto), mismo patrón de datos ya calculados.
Reglas del juego (FIVB) y uso de la app	"¿Por qué no me deja hacer un tercer cambio de líbero?" / "¿Cómo exporto un partido?"	Bajo, si se ancla al texto del manual (<code>manual_usuario_rallystats.pdf</code>) en vez de dejarlo "recordar" reglas de memoria.
Lectura táctica	"¿Qué zona de ataque rindió menos en el segundo set?" / "¿Contra qué rotación del rival nos costó más recibir?"	Alto - requiere interpretar patrones sobre el log de RallyEvent, no solo repetir un número, con margen real de sobre-interpretación en una muestra chica. Dejar para una segunda etapa, con avisos de que es orientativa.

Cómo se "entrena" en realidad

La palabra "entrenar" lleva a pensar en fine-tuning (ajustar los pesos de un modelo con ejemplos propios), pero no es lo que conviene ni lo que se necesita acá:

- **No hay volumen de datos para entrenar nada propio.** Fine-tuning tiene sentido con miles/millones de ejemplos; acá cada club tiene, como mucho, algunas decenas de partidos guardados localmente en Hive.

- **Los datos son por dispositivo, no centralizados.** Hoy no existe ningún lugar donde junten los partidos de todos los usuarios - habría que construir eso desde cero solo para "entrenar", lo cual es un proyecto en sí mismo y no aporta nada que el enfoque de abajo no resuelva mejor.
- **Lo que hace falta es contexto por consulta (grounding), no entrenamiento.** Cada vez que el usuario pregunta algo, se arma un mensaje que incluye: el MatchStats del partido en cuestión, el roster, y - si la pregunta es de reglas - el texto relevante del manual. El modelo no "aprende" el partido de antemano, lo lee en el momento, como si le pasaras un resumen a upa cada vez.

Esto es el patrón conocido como **RAG** (retrieval-augmented generation) simplificado: en vez de un modelo que "sabe" de memoria los datos, se le da el material necesario junto con la pregunta. Dado el tamaño de los datos por partido (unas decenas de jugadores y unos pocos cientos de eventos por set), ni siquiera hace falta un sistema de búsqueda/embeddings sofisticado - alcanza con armar el contexto a mano según qué tipo de pregunta se detecte, o directamente mandar siempre el MatchStats del partido abierto.

Tres formas posibles de diseñarlo

Opción	Cómo funciona	Particularidad
A. LLM en la nube (API tipo Claude/OpenAI)	La app manda la pregunta + el contexto (stats ya calculadas) a un backend propio, que llama a la API del proveedor y devuelve la respuesta.	Mejor calidad de respuesta por lejos, entiende lenguaje libre y ambiguo. Requiere backend nuevo y conexión a internet en el momento de usarlo.
B. Modelo chico on-device (ej. Gemma/Llama compilado en la app)	Un modelo pequeño corre embebido en el propio dispositivo, sin salir a internet.	Mantiene el offline-first, pero la calidad de respuesta baja mucho, el tamaño de la app crece fuerte, y consume batería/rendimiento - un problema concreto porque el público real usa celulares de gama media/baja durante un partido en vivo.
C. Reglas + templates (sin LLM real)	Un set fijo de preguntas frecuentes con respuestas armadas por template, rellenas con los datos del partido.	100% offline, gratis, cero riesgo de que invente algo. A cambio, no es realmente conversacional: es un buscador con otra piel, y solo cubre lo anticipado de antemano.

Ninguna opción es estrictamente superior: A da la mejor experiencia pero es la que más se aleja del offline-first actual; C es la más alineada con la filosofía de la app pero deja de ser un "chatbot" en el sentido que probablemente se busca; B intenta un punto medio pero con compromisos serios de calidad y peso de la app.

Qué hay que construir, paso a paso (tomando la Opción A)

1. **Backend proxy propio** (ej. una Cloud Function, similar en espíritu a cómo ya se depende de Firebase para el login). Necesario porque la API key del proveedor de IA no puede viajar dentro del APK - cualquiera podría extraerla y usarla a costa de la cuenta.
2. **Definir qué datos viajan como contexto:** lo mínimo indispensable (el MatchStats ya calculado), no el log crudo completo de eventos, para no exponer de más y para no gastar tokens de más.
3. **Pantalla nueva** en lib/screens/ (por ejemplo chatbot/), con historial de conversación, accesible desde el resumen de un partido en matches/ o desde el menú principal.
4. **System prompt cuidado:** instrucciones explícitas de no inventar números que no estén en el contexto provisto, responder en español, y ceñirse a terminología de vóley/FIVB correcta.
5. **Gating por suscripción:** reusar SubscriptionController.isPremium, igual que ya se hace con pizarra/estadísticas/tope de partidos.

6. **Manejo de sin-conexión:** mensaje claro cuando no hay internet, en vez de spinner infinito o error genérico (la app ya tiene esa lógica para el login; se puede reusar el patrón).
7. **Control de costo:** límite de consultas por mes o por partido - a diferencia de RevenueCat (que es un % fijo conocido de antemano), acá el gasto depende directamente de cuánto lo use cada usuario.

Cómo encaja con lo que ya existe

El proyecto ya tiene un precedente exacto para esta discusión: hoy la app es "offline para todos los datos de juego" con dos excepciones explícitas y deliberadas - login (control de dispositivos) y suscripción premium (Google Play Billing). Un chatbot en la nube sería una **tercera excepción** al mismo principio, y conviene tratarla con el mismo criterio: acotada, justificada, y clara para el usuario sobre qué datos salen y cuándo.

- Encaja naturalmente como **feature premium**: la infraestructura de SubscriptionController ya existe para trabar funciones específicas sin un gate central, así que sumar una más es consistente con el patrón actual.
- El **manual de usuario** (`tool/generate_manual.dart`) es contenido reusable directo como material de referencia para las preguntas de reglas/uso - no hay que redactarlo de nuevo.
- Los **reportes en PDF** (`pdf_report_service.dart`) y el chatbot resolverían necesidades parecidas (comunicar el partido) por vías distintas - uno es un documento fijo, el otro una consulta puntual. No se reemplazan, se complementan.

Pros y contras

A favor

- Diferencial fuerte para justificar la suscripción premium
- Vuelve accesible el manual/reglas sin tener que buscar en un PDF
- Resúmenes en lenguaje natural listos para compartir con el equipo
- Baja la curva de aprendizaje para usuarios nuevos
- Reusa datos que la app ya calcula - no hay que generar nada nuevo

En contra

- Rompe el principio de offline-first para datos de juego
- Costo variable por uso, difícil de prever de antemano
- Backend nuevo que mantener, con su propia seguridad y monitoreo
- Riesgo de respuesta incorrecta (alucinación) sobre reglas o números
- Dependencia nueva de un proveedor externo de IA (precios, disponibilidad)

Riesgos a tener en cuenta especialmente

Alucinaciones en un contexto de partido real

PRIORIDAD ALTA

Si el chatbot inventa o confunde una estadística durante un partido en vivo (por ejemplo, una eficacia de recepción mal calculada), el usuario puede tomar una decisión táctica equivocada confiando en eso. Mitigación: nunca dejar que el modelo calcule un número - solo que lo lea de un MatchStats ya armado y lo redacte; para explicaciones de reglas, anclarlo al texto del manual en vez de dejarlo "recordar" reglas FIVB de memoria.

Datos de menores de edad

PRIORIDAD ALTA

El vóley se juega mucho en categorías juveniles - es probable que varios equipos cargados en la app tengan jugadores menores. Mandar nombres y estadísticas de chicos a un proveedor externo de IA es un tema a resolver conscientemente (anonimizar nombres antes de mandarlos al LLM es una opción simple: mandar "Jugador #4" en vez del nombre real, y que la respuesta se re-mapee del lado de la app).

Costo variable sin techo

PRIORIDAD MEDIA

A diferencia de RevenueCat, un LLM se cobra por uso. Sin un límite duro de consultas, un usuario (o un bug con reintentos) podría generar un costo desproporcionado. Mitigación: límite mensual por cuenta, cacheo de respuestas repetidas, y contexto acotado (mandar solo lo necesario, no el partido entero crudo).

Complejidad de mantenimiento PRIORIDAD MEDIA

Suma una tercera pieza de infraestructura externa (después de Firebase y RevenueCat), cada una con su propio panel, sus propias claves y su propio modo de fallar. Para un proyecto de un solo desarrollador, esto es un costo real de atención continua, no solo de implementación inicial.

Esfuerzo estimado

Bloque de trabajo	Esfuerzo relativo
Backend proxy + integración con la API del LLM	Medio
Armado del contexto (grounding) desde stats_engine y el manual	Medio
UI de chat nueva (pantalla, historial, estados de carga/error)	Bajo
Gating premium + control de costo/límites	Bajo
Anonimización de datos de jugadores antes de mandarlos	Medio
Lectura táctica sobre eventos crudos (caso ambicioso, fase 2)	Alto

En conjunto, es una feature de porte comparable a haber sumado la suscripción premium con RevenueCat - no un ajuste incremental como los que suelen verse en el historial reciente del proyecto, sino un desarrollo de punta a punta con una pieza de backend nueva.

Recomendación

Empezar acotado: Opción A, pero limitada a "explicame esta estadística / esta regla" sobre el partido ya abierto - no un chat abierto sin límites - gateado a premium, con nombres de jugadores anonimizados antes de salir del dispositivo, y un tope duro de consultas mensuales para no exponerse a un costo variable impredecible. Recién si eso valida que el uso real justifica la inversión, ampliar hacia comparaciones entre partidos y lectura táctica.

8. Otras funcionalidades adicionales

Ideas no cubiertas arriba, con una nota breve de cómo encajarían (sin el mismo nivel de detalle porque son de menor prioridad o dependen de decisiones previas):

- **Gráfico de progresión del marcador dentro de un set:** el dato ya existe (`RallyEvent.ownScoreAfter/rivalScoreAfter` se guarda en cada evento), sólo falta visualizarlo - un gráfico de líneas en `MatchSummaryScreen`. Requiere sumar un paquete de gráficos (el proyecto no tiene ninguno todavía; `fl_chart` es una opción estándar en Flutter). Complejidad baja-media.
- **Comparador de dos partidos lado a lado:** reutiliza `StatsEngine.compute()` para dos `VolleyMatch` y muestra sus `MatchStats` en columnas contiguas. Complejidad baja, ya que no requiere agregación nueva (a diferencia del panel de temporada, son solo dos partidos puntuales elegidos a mano).

- **Reportes de PDF personalizables:** elegir qué secciones/columnas incluir en el PDF que ya genera `pdf_report_service.dart`, en vez del formato fijo actual. Complejidad media (UI de selección + parametrizar el armado del PDF existente).
- **Notas de scouting en texto libre por partido:** un campo notes simple en VolleyMatch (mismo patrón de campo nuevo opcional). Complejidad muy baja, valor más bien secundario.
- **Reproducir/animar una jugada de pizarra guardada:** `Play.strokes` ya guarda los puntos de cada trazo en orden; se podría animar el dibujo trazo por trazo en vez de mostrarlo estático. Requiere trabajo de UI en `WhiteboardPainter`. Complejidad media, y es más una mejora de la pizarra (que ya es premium) que un feature nuevo para vender aparte.

Ideas descartadas para este documento por alcance desproporcionado respecto al resto de la app (mencionadas para que quede registro de que se consideraron): edición colaborativa en vivo de un mismo partido desde dos dispositivos a la vez, y video-tagging (sincronizar eventos con un video del partido) - ambas requerirían infraestructura en tiempo real o de archivos pesados que no tiene relación con nada de lo que ya existe en el proyecto.

9. Orden sugerido de implementación

No es un compromiso, es una sugerencia según esfuerzo vs. impacto, para discutir:

1. **Exportar a CSV** (§4) - el más rápido, prácticamente sin riesgo, reusa todo lo que ya existe.
2. **Fixture con autocompletado** (§6) - complejidad media, no toca reglas de juego, y da un motivo de uso frecuente de la app entre partido y partido.
3. **Panel de temporada** (§1) y **Scouting de rival fase 1** (§2) - comparten el mismo motor de agregación, tiene sentido hacerlos juntos o uno inmediatamente después del otro.
4. **Estadísticas por rotación** (§3) - sí requiere tocar `MatchController/RallyEvent`, dejarlo para cuando los tres anteriores ya estén asentados y probados.
5. **Backup en la nube** (§5) - mayor esfuerzo e implicancias de producto; conversar el alcance (manual vs. automático, política de conflictos, costo) antes de empezar a codear.
6. **Chatbot conversacional** (§7) - el de mayor porte de todo el documento (comparable a haber sumado `RevenueCat`); no arrancar sin antes acordar alcance acotado, presupuesto de costo variable, y política de datos de menores.